Learned or chosen features? Feed-forward neural networks for function fitting and digit recognition

Rasul Ruslanovitsj Øzdber[1, *]

[1] *Department of Physics, University of Oslo, Norway*

(Dated: November 9, 2026)

Neural networks learn their features from the data, whereas linear models use features chosen by the analyst. We study when learning the features justifies the higher cost of training. We wrote a feed-forward neural network from scratch and verified it against automatic differentiation and closed-form solutions. For 100 noisy observations of Runge's function, sigmoid networks with five or more hidden nodes had a 1.5 to 2.8 times smaller mean-square deviation from the true function than the best polynomial fit, because the function's nearby complex poles make powers of x an inefficient basis. For a smooth function without such poles, or with more data and fixed network settings, polynomials were more accurate and far cheaper. On MNIST handwritten digits, one hidden layer of 512 nodes reached 98.0% test accuracy, against 92.8% for logistic regression. Training depended strongly on the optimizer, the activation function and, on MNIST, the scaling of inputs and weights, whereas L_1 and L_2 penalties did not improve on early stopping. Learned features thus pay off when no good basis is known or the obvious one fits poorly.

I. INTRODUCTION

Every supervised learning method describes its input through a set of features. In linear regression the features are chosen by the analyst, for example the powers of x in a polynomial fit, and the parameters follow from a closed-form solution. A neural network instead learns its features from the raw input during training. This removes the need to choose a basis, but turns the fit into a non-convex optimization problem with many hyperparameters and no closed-form solution to compare with. We ask when this trade-off favours the network.

In principle, neural networks are flexible enough. A single hidden layer can approximate any continuous function on a compact domain to arbitrary accuracy [1, Sec. 8.4]. Being able to represent a function does not, however, mean that training will find it. Deep sigmoid networks were long limited by vanishing gradients, and careful initialization [2, 3], rectified activation functions [4, 5] and adaptive optimizers [6, 7] were needed to make them trainable [8, 9].

We study two problems at opposite ends of this question. For Runge's function, a function of one variable, polynomials are the natural features. In earlier work we fitted ordinary least squares (OLS), Ridge and Lasso polynomials to 100 noisy observations of it and selected them by cross-validation [10]. These fits are benchmarks on the same data, and since the true function is known, the error of every model against it can be measured directly. For the MNIST (Modified National Institute of Standards and Technology) handwritten digits [11, 12], with 784 pixels per image, no good hand-crafted features are known. Multinomial logistic regression is the baseline there, and published dense and convolutional networks give further reference points [12, 13].

We address four questions. Is a network more accurate than the best polynomial fitted to the same data, and if so, why? How do the optimizer, the learning rate, the batch size, the activation function, the depth and the width affect training? Do L_1 and L_2 penalties improve networks in the way Lasso and Ridge improve polynomial fits? And how much do hidden layers improve on logistic regression for MNIST, and which errors remain?

The network, backpropagation, training and experiments are implemented from scratch for this work (Sec. III C), and the use of a large language model (LLM) in writing the code and the text is declared in Appendix D. The gradient methods are reused from our earlier study [10], and scikit-learn [14] provides the scaling, cross-validation, metrics and reference models.

Section II presents the network, backpropagation, the optimizers and the polynomial approximation of Runge's function, and Sec. III the data, the implementation and its verification, and the protocol for selecting and evaluating the models. The results are presented in Sec. IV and discussed in Sec. V, and Sec. VI gives the conclusions, the limitations and further work. All code, saved results and notebooks are available at `https://github.com/rasuluzd/FYS-STK-Project2`.

II. THEORY

A. Feed-forward neural networks

A feed-forward neural network (FFNN) is a composition of layers. Layer 0 holds the input, layer L the output, and the layers in between are called hidden. Layer l has M_l nodes. Each node forms a weighted sum of the outputs of the previous layer, adds a bias and applies an

* Parts of the text and code were produced with the assistance of a large language model; see Appendix D.

activation function f [1, Sec. 8.5],

$$z_j^l = \sum_{i=1}^{M_{l-1}} w_{ij}^l a_i^{l-1} + b_j^l, \qquad a_j^l = f\left(z_j^l\right), \tag{1}$$

where $a_i^0 = x_i$ are the inputs. The weights w_{ij}^l and the biases b_j^l are the parameters of the network, collectively denoted Θ. For n samples stored as the rows of $\boldsymbol{A}^{l-1}$, a layer computes $\boldsymbol{Z}^l = \boldsymbol{A}^{l-1}\boldsymbol{W}^l + \boldsymbol{b}^l$, where $\boldsymbol{W}^l$ is an $M_{l-1} \times M_l$ matrix and the bias vector is added to every row. We denote an architecture by its layer sizes: 1-50-1, for example, is a network with one input, one hidden layer of 50 nodes and one output.

Without hidden layers, the network reduces to a linear model followed by the output activation. Each hidden layer computes new features from those of the layer below, and the output layer is a linear model in the features of the last hidden layer.

B. Cost functions and regularization

Training minimizes a cost function $C(\Theta)$ averaged over the samples. With $\boldsymbol{y}_i$ the target of sample i and $\boldsymbol{a}_i^L$ the network output, we consider three costs, each paired with the output activation that matches the range of the target [1, Secs. 5.3, 5.6 and 8.7].

For regression, the output is linear, $\boldsymbol{a}^L = \boldsymbol{z}^L$, and the cost is half the mean squared error (MSE),

$$C_{\mathrm{MSE}} = \frac{1}{2n}\sum_{i=1}^{n} \left\|\boldsymbol{a}_i^L - \boldsymbol{y}_i\right\|^2, \quad \frac{\partial C_{\mathrm{MSE}}}{\partial \boldsymbol{a}_i^L} = \frac{1}{n}\left(\boldsymbol{a}_i^L - \boldsymbol{y}_i\right). \tag{2}$$

The factor $\frac{1}{2}$ only removes a factor of two from the gradient; all errors reported below are the MSE itself.

For binary classification, a single sigmoid output is interpreted as the probability of class 1, and the cost is the binary cross entropy with targets $y_i \in \{0, 1\}$,

$$\begin{aligned} C_{\mathrm{BCE}} &= -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \ln a_i^L + (1-y_i)\ln\left(1-a_i^L\right)\right], \\ \frac{\partial C_{\mathrm{BCE}}}{\partial a_i^L} &= \frac{1}{n}\,\frac{a_i^L - y_i}{a_i^L\left(1-a_i^L\right)}. \end{aligned} \tag{3}$$

For classification into K classes, the output layer has K nodes with the softmax activation, whose outputs are positive and sum to one. The cost is the multiclass cross entropy with one-hot targets, where $y_{ik} = 1$ if sample i belongs to class k and zero otherwise,

$$\begin{aligned} a_{ik}^L &= \frac{\exp(z_{ik}^L)}{\sum_{m=1}^{K}\exp(z_{im}^L)}, \\ C_{\mathrm{CE}} &= -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\ln a_{ik}^L, \\ \frac{\partial C_{\mathrm{CE}}}{\partial a_{ik}^L} &= -\frac{1}{n}\,\frac{y_{ik}}{a_{ik}^L}. \end{aligned} \tag{4}$$

Each cost can be augmented by a penalty $\lambda\,\Omega(\Theta)$, with

$$\Omega_2 = \sum_{\theta\in\Theta}\theta^2, \qquad \Omega_1 = \sum_{\theta\in\Theta}|\theta|, \tag{5}$$

where θ runs over all weights and biases. The L_2 and L_1 penalties add $2\lambda\theta$ and $\lambda\,\mathrm{sign}(\theta)$ to the gradient, with $\mathrm{sign}(0) = 0$, a valid subgradient at the kink of $|\theta|$. They affect only the training, and the reported errors never include them. They play the same role for networks as Ridge and Lasso regression do for linear models [1, Secs. 3.8, 3.9 and 8.12].

C. Activation functions

The hidden layers use one of three activation functions [1, Sec. 8.6],

$$\begin{aligned} \sigma(z) &= \frac{1}{1+e^{-z}}, & \sigma'(z) &= \sigma(z)\left[1-\sigma(z)\right], \\ \mathrm{ReLU}(z) &= \max(0, z), & \mathrm{ReLU}'(z) &= \begin{cases} 1 & z > 0 \\ 0 & z \le 0 \end{cases}, \\ \mathrm{LReLU}(z) &= \max(\alpha z, z), & \mathrm{LReLU}'(z) &= \begin{cases} 1 & z > 0 \\ \alpha & z \le 0 \end{cases}. \end{aligned} \tag{6}$$

The sigmoid saturates for large $|z|$, and its derivative never exceeds $\frac{1}{4}$. The rectified linear unit (ReLU) has unit derivative for every positive input [4], but a ReLU node whose input is negative for all samples receives no gradient and stops learning. The leaky ReLU (LReLU) avoids this with a small slope $\alpha = 0.01$ for negative inputs [5]. The value of the derivative at $z = 0$ is a convention.

D. Backpropagation

Backpropagation [1, Sec. 8.8] computes the gradient of the cost with respect to every parameter by applying the chain rule layer by layer to the error $\delta_j^l = \partial C/\partial z_j^l$ of each node. This gives four equations [1, Sec. 8.8.4],

$$\boldsymbol{\delta}^L = f'\left(\boldsymbol{z}^L\right) \circ \frac{\partial C}{\partial \boldsymbol{a}^L}, \tag{7}$$

$$\boldsymbol{\delta}^l = \left(\boldsymbol{W}^{l+1}\boldsymbol{\delta}^{l+1}\right) \circ f'\left(\boldsymbol{z}^l\right), \tag{8}$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l, \tag{9}$$

$$\frac{\partial C}{\partial w_{ij}^l} = \delta_j^l\, a_i^{l-1}, \tag{10}$$

where $\circ$ is the elementwise product and the vectors refer to a single sample. Equation (7) holds for an output activation that acts on each node separately; for the softmax, f' is replaced by its full Jacobian (Appendix A). A forward pass stores all $\boldsymbol{z}^l$ and $\boldsymbol{a}^l$, and a backward pass from

$l = L$ down to $l = 1$ then evaluates Eqs. (7)–(10) and sums the contributions of all samples. These equations give the gradient of the data term; the penalty gradient of Sec. II B is added to it.

Inserting the costs into Eq. (7) shows why the output activation and the cost should be chosen as a pair. For each of the three pairs above, the derivative of the activation cancels that of the cost, and the output error of a sample reduces to $\boldsymbol{\delta}^L = \left(\boldsymbol{a}^L - \boldsymbol{y}\right)/n$ (Appendix A). A sigmoid or softmax output combined with the squared error retains the factor f', which reduces the gradient in general and lets it vanish when the network is confidently wrong.

Backpropagation is reverse-mode automatic differentiation applied to a layered function [1, Sec. 4.14.4]. With two matrix products per layer in the backward pass, for Eqs. (8) and (10), and one in the forward pass, a gradient costs about two to three forward passes, whereas central finite differences need two per parameter.

E. Vanishing gradients and initialization

Applying Eq. (8) repeatedly from the output down to layer l gives the product

$$\boldsymbol{\delta}^l = \operatorname{diag}\big(f'(\boldsymbol{z}^l)\big)\boldsymbol{W}^{l+1}\operatorname{diag}\big(f'(\boldsymbol{z}^{l+1})\big)\boldsymbol{W}^{l+2}\cdots\boldsymbol{W}^L\boldsymbol{\delta}^L, \tag{11}$$

where each $\boldsymbol{W}^k$ acts as in Eq. (8). The gradient that reaches the first layers can therefore shrink or grow exponentially with the depth [1, Sec. 8.9]; for the sigmoid, every factor contains a derivative of at most $\frac{1}{4}$.

The initial weights determine where this product starts. For weights independent of the inputs and of each other, with zero mean and variance $\operatorname{Var}(w)$, Eq. (1) gives $\operatorname{Var}(z) = M_{l-1}\operatorname{Var}(w)\,\mathrm{E}[a^2]$, with $\mathrm{E}[a^2]$ the mean square of the inputs to the layer. For inputs with zero mean, such as standardized data but not sigmoid outputs, $\mathrm{E}[a^2] = \operatorname{Var}(a)$, the form of the lecture notes [1, Sec. 8.10]. A signal of constant size through the layers thus requires $\operatorname{Var}(w) \propto 1/M_{l-1}$ [2]. We draw the weights from normal distributions with zero mean and variance $1/M_{l-1}$ for sigmoid and output layers, and $2/M_{l-1}$ for the ReLU family, which sets about half of its inputs to zero [3]. Larger weights, and unscaled inputs, drive sigmoid nodes into saturation from the start. The biases are set to 0.01, which nudges ReLU nodes towards the active side of the kink.

F. Gradient-based optimization

The parameters are updated with the gradient methods of our earlier study [10]. Gradient descent (GD) uses the gradient $\boldsymbol{g}_t$ of the cost over all samples and updates $\Theta_{t+1} = \Theta_t - \gamma\boldsymbol{g}_t$, where γ is the learning rate. Stochastic gradient descent (SGD) [1, Sec. 4.7] applies the same rule to minibatches of B samples, reshuffled in every epoch, that is, every pass through the data. We call GD and SGD with a fixed learning rate plain, to distinguish them from the adaptive methods RMSprop (root mean square propagation) [6] and Adam (adaptive moment estimation) [7], which give each parameter its own step size from a running average of its squared gradient,

$$\begin{aligned}
\text{RMSprop:}\quad \boldsymbol{s}_t &= \beta\,\boldsymbol{s}_{t-1} + (1-\beta)\,\boldsymbol{g}_t^2,\\
\Theta_{t+1} &= \Theta_t - \frac{\gamma\,\boldsymbol{g}_t}{\sqrt{\boldsymbol{s}_t} + \epsilon},\\
\text{Adam:}\quad \boldsymbol{m}_t &= \beta_1\boldsymbol{m}_{t-1} + (1-\beta_1)\boldsymbol{g}_t,\\
\boldsymbol{v}_t &= \beta_2\boldsymbol{v}_{t-1} + (1-\beta_2)\boldsymbol{g}_t^2,\\
\Theta_{t+1} &= \Theta_t - \gamma\,\frac{\boldsymbol{m}_t/(1-\beta_1^t)}{\sqrt{\boldsymbol{v}_t/(1-\beta_2^t)} + \epsilon},
\end{aligned} \tag{12}$$

with all operations elementwise, $\beta = 0.99$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. Dividing by the root mean square of recent gradients gives Adam steps of order γ whatever the size of the gradient. Since the cost of a network is not convex, the learning rate, the initial weights and the batch size can decide which minimum is reached, or whether training progresses at all [1, Sec. 8.15].

G. Polynomial approximation of Runge's function

Runge's function $g(x) = 1/(1 + 25x^2)$ is smooth on $[-1, 1]$, yet its polynomial approximations converge slowly, even without noise. For a function that is analytic inside the Bernstein ellipse with parameter $\rho > 1$, the error of the best polynomial approximation of degree d decreases at the geometric rate ρ^{-d} [15, Chap. 8]. The poles of g at $x = \pm\mathrm{i}/5$ limit this ellipse to $\rho = (1+\sqrt{26})/5 \approx 1.22$, so the squared error falls on average by a factor $\rho^{-2} \approx 0.67$ per degree; since g is even, only the even degrees improve its best approximation, each by about $\rho^{-4} \approx 0.45$. A function without singularities in the complex plane, such as $\cos 3x$, is approximated with an error that decreases faster than ρ^{-d} for every ρ [15, Chap. 8]. These rates hold in any basis; a better-conditioned basis, such as the Chebyshev polynomials, improves only the numerical stability of the fit. Runge's function is therefore a demanding test for polynomial features, and $\cos 3x$ a favourable one.

III. METHODS

A. Data

Function fitting. The regression data are the 100 observations of our earlier study [10], $y_i = g(x_i) + \varepsilon_i$, with x_i uniform on $[-1, 1]$ and Gaussian noise of standard deviation $\sigma_\varepsilon = 0.1$, generated with NumPy's default generator (PCG64) and seed 2026. Two further sets of 2000 observations, with seeds 2027 and 2028, serve as test sets. The first is the test set of our earlier study and was also read

by a few pilot runs before the final test (Sec. VI); the second was drawn for this study and is read only by the final evaluation. The network receives x itself, standardized with the training mean and standard deviation, and the target is centred with the training mean.

Digit recognition. The MNIST data set [11] has 60 000 training and 10 000 test images of handwritten digits with 28×28 grey-scale pixels, flattened to 784 values and divided by 255; the labels are one-hot encoded. A stratified split of the training images with scikit-learn's `train_test_split` (Mersenne Twister, seed 2026) gives 50 000 images for fitting and 10 000 for validation, on which all choices are based. The test images are used only at the end, by the selected models refitted on all 60 000 training images. In both problems, the initial weights and the minibatch order come from PCG64 generators with seeds from 2026 upwards, one per initialization.

B. Reference models

For Runge's function the references are the polynomial fits selected by cross-validation in our earlier study [10]: OLS of degree 12, Ridge of degree 12 with $\lambda = 10^{-10}$, and Lasso of degree 15 with $\lambda = 10^{-4}$. Their features $x, \ldots, x^d$ are chosen by hand, and the OLS and Ridge parameters follow from closed-form solutions. That study used the cost $\mathrm{MSE} + \lambda\Omega$, so its λ corresponds to 2λ here. For MNIST the reference is multinomial logistic regression, which is the network without hidden layers. We also compare with the neural network and the logistic regression of scikit-learn [14].

C. Implementation

The network, backpropagation and training are implemented in NumPy [16]. All weights and biases are stored in one parameter vector, so the gradient has the same form as for the linear models of our earlier study [10], whose update rules and minibatch loop are reused unchanged. The number of layers and nodes, the activation of each layer, the cost, the penalty and the initialization are all arguments of one class. The core of the backward pass reads

```
delta = output_error(...) / n
for l in reversed(range(L)):
    dW[l] = a[l].T @ delta + penalty(W[l])
    db[l] = delta.sum(axis=0) + penalty(b[l])
    if l > 0:
        delta = (delta @ W[l].T) * df[l-1](z[l-1])
```

where `a[l]` is the output of layer l, with the samples in rows, and `W[l]` holds the weights from layer l to layer $l+1$.

Scikit-learn [14] provides the scaling, the cross-validation folds, the grid search, the classification metrics and the reference models; the network is wrapped as a scikit-learn estimator, so that the scaler is refitted within every fold. Autograd [17] is used in the tests only, and the figures are made with Matplotlib [18] and seaborn [19].

TABLE I. Checks of the implementation against independent references. All pass within the stated accuracy.

Check	Result
Backpropagation against Autograd on a separately written cost, for all activations, costs and penalties	Equal to rounding error
Backpropagation against central finite differences	Equal within the accuracy of the difference scheme
No hidden layer, linear output, on polynomial features	Recovers the exact OLS and Ridge coefficients
No hidden layer, softmax output	Same probabilities as logistic regression in scikit-learn, with and without penalty
1-50-1 sigmoid network with the seeds and settings of a published reference implementation [20]	Reproduces its printed training error and five-fold errors to the last printed digit
Polynomial models	Reproduce the results of our earlier study [10]
One hidden layer on small digit images, against scikit-learn	Test accuracy above 90% and within three percentage points of scikit-learn's

D. Verification

An incorrect gradient does not prevent a network from training; it only makes it train worse. The code was therefore checked against independent references before it was used (Table I). Every result in this report is written to file by scripts with fixed seeds, and all figures and tables are drawn from these files.

The measured cost of a gradient agrees with Sec. II D: for networks from 151 to more than half a million parameters, one gradient took between one and four forward passes.

E. Model selection and evaluation

The settings for Runge's function are chosen by ten-fold cross-validation (CV) on the 100 observations [1, Sec. 2.12], with the same folds for networks and polynomials (scikit-learn's `KFold`, Mersenne Twister, seed 2026) and the scaling fitted within each fold. The CV error is the mean validation MSE over the folds, with the standard error (SE) $s/\sqrt{10}$, where s is the standard deviation of the ten fold errors; this SE is somewhat too small, since

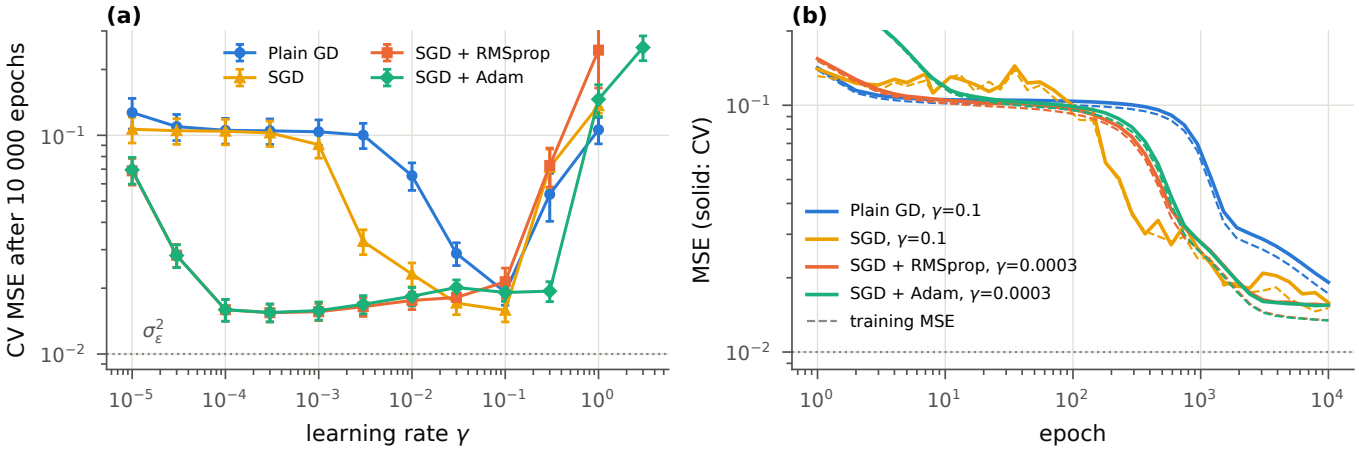


FIG. 1. Adam and RMSprop reach the lowest cross-validation (CV) error over a much wider range of learning rates than plain SGD, and plain GD has not converged. The 1-50-1 sigmoid network on the 100 observations of Runge's function, with the same budget of 10 000 epochs for every method. (a) Ten-fold CV MSE at the end of training against the learning rate, with standard errors; missing points diverged. (b) CV (solid) and training (dashed) MSE against epoch at each method's best rate. The dotted line marks the noise variance σ_ε^2.

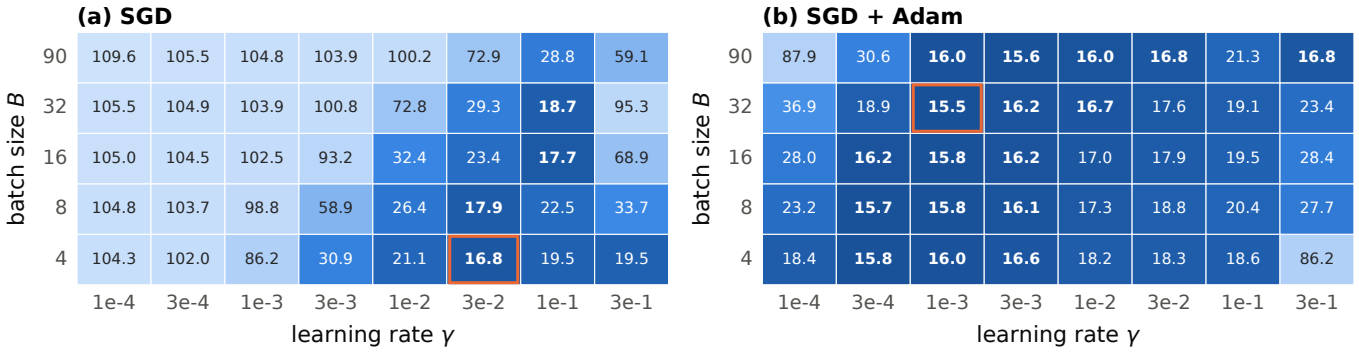


FIG. 2. For plain SGD, the best batch sizes and learning rates lie along a diagonal, whereas Adam is nearly insensitive to both. CV MSE ($\times 10^3$) of the 1-50-1 sigmoid network after 3000 epochs, against the batch size and the learning rate, for (a) plain SGD and (b) Adam. Darker is better; the square marks the lowest value, and bold numbers are within one standard error of it. A batch of 90 is all the training samples of a fold.

the training parts of the folds overlap. Because the lowest CV error in a grid is also optimistic, we apply the one-standard-error rule: among all settings within one SE of the best, we choose the most regularized one, that is, the smallest network or the largest penalty [21, Sec. 7.10]. For MNIST, the same rule is applied to the validation accuracy with its binomial standard error. Early stopping, that is, ending the training at the epoch with the best validation score [1, Sec. 8.12], is treated as one more setting, and unless stated otherwise the minibatches contain 16 samples for Runge's function and 128 for MNIST.

Since g is known, we also measure how far a fitted model $\hat{g}$ is from the function itself, $\mathrm{MSE}_g = \frac{1}{m}\sum_i [\hat{g}(x_i) - g(x_i)]^2$, on the $m = 2000$ test inputs. This error against the true function excludes the noise in the test targets, which no model can predict, but it still reflects the noisy training data, the finite sample and the training.

For classification we report the accuracy, the fraction of correctly classified images, with the binomial standard error $\sqrt{p(1-p)/n_t}$ for an accuracy p on n_t test images, about 0.15 percentage points here. The confusion matrix C_{ij} counts the images of true class i predicted as class j. The recall of a class is its diagonal element divided by the row sum, the fraction of its images that are found; the precision is the diagonal element divided by the column sum, the fraction of its predictions that are correct; and the F_1 score is their harmonic mean [1, Sec. 5.12]. The networks are trained on the cross entropy, since the accuracy changes in steps and has zero gradient almost everywhere.

IV. RESULTS

A. Regression of Runge's function

1. Optimizer, learning rate and batch size

The choice of optimizer mainly determined how sensitive the training was to the learning rate (Fig. 1a). With the same number of passes through the data, Adam and RMSprop reached the lowest CV error for every learning rate from 10^{-4} to 3×10^{-3}, plain SGD only between 0.03 and 0.1, and plain GD, with one update per epoch, had not converged even at its best rate. At larger rates the error grew, gradually for the adaptive methods, until it reached that of predicting the mean or the training diverged. At their best rates the three minibatch methods ended within one standard error of each other, so the adaptive methods offered robustness to the learning rate, not a better minimum. Every method first spent a hundred or more epochs predicting little more than the mean before it formed the peak, and larger steps shortened this plateau (Fig. 1b).

For plain SGD, the batch size and the learning rate acted as a single parameter (Fig. 2a): a smaller batch gives more updates per epoch and worked best with a smaller learning rate. Adam was nearly insensitive to both (Fig. 2b), as expected from its normalized steps.

2. Activation function and depth

With Adam, the architecture had little influence on the fit (Fig. 3, bottom): all networks from one hidden layer of five nodes to five layers of 100 nodes reached similar CV errors. The sigmoid was the best activation here. ReLU networks, which are piecewise linear and need many kinks to follow the smooth peak, had 4–30% higher CV errors than sigmoid networks of the same size, and the leaky slope made no systematic difference.

With plain SGD, in contrast, sigmoid networks with four or five hidden layers did not train at any of the learning rates tried (Fig. 3, top): their CV and training errors stayed near those of predicting the mean, a failure of the training rather than overfitting. ReLU networks trained at every depth.

The gradients in each layer show why (Fig. 4). In an untrained deep sigmoid network, the gradient decreased by about a factor of four per layer towards the input, as Eq. (11) predicts, while ReLU layers passed it on unchanged. Plain SGD therefore left the weights of the first layers almost unchanged, whereas Adam changed every layer substantially (Appendix C). Weights drawn from $\mathcal{N}(0, 1)$ instead of $\mathcal{N}(0, 1/M_{l-1})$ did not make the gradient vanish at this width, because the larger weights compensate for the smaller derivatives, but they saturate the nodes (Sec. IV B 3).

3. Overfitting, early stopping and penalties

Without a penalty, large networks overfitted in the same way as high-degree polynomials [10]: trained for long, their training error fell below the noise variance while the CV error increased (Fig. 5). Networks with a single hidden layer hardly overfitted, even with 100 nodes. Early stopping removed almost all of the effect: at the epoch with the lowest mean CV error, every sigmoid network from 7 to more than 20 000 parameters was within one SE of the best.

Penalties brought no further improvement (Fig. 6). For both L_2 and L_1, every penalty up to $\lambda = 10^{-4}$ was within one SE of no penalty, 10^{-3} destroyed the fit of the networks shown, and larger penalties left only the mean. The one-standard-error rule selected $\lambda = 10^{-4}$ for both. Penalizing the biases or not changed the CV error by much less than its SE, and a stronger L_1 penalty drove most output weights towards zero, much as the Lasso does with polynomial coefficients (Appendix B). The L_2-penalized network was about as accurate as Ridge and the L_1-penalized network more accurate than Lasso, but neither, trained for a fixed 3000 epochs, improved on the early-stopped unpenalized networks (Table II); penalties combined with early stopping were not tested on this problem.

4. Comparison with polynomial regression

The selected networks fitted Runge's function more closely than the polynomials, but by a margin that is small compared with the noise (Table II). Every unpenalized sigmoid network with five or more hidden nodes had lower errors than the polynomials on all three measures, yet their CV errors overlap within the SEs, so on 100 noisy observations the CV errors alone cannot rank the models. In a paired comparison on the same folds, the 1-50-1 network had the lower error by about two standard errors of the difference; since the folds share training data, this is an indication rather than a significance test.

The independent observations separate the models more clearly, because MSE_g removes the noise floor of the test targets. Against the true function, these networks had a 1.5 to 2.8 times smaller error than OLS, and on the second test set every network again had a lower test error than OLS, in almost the same order. Even the network that the one-standard-error rule selects, with two sigmoid nodes and seven parameters, was about as accurate as OLS with thirteen. Neither the ReLU network nor the penalized networks improved on the unpenalized sigmoid networks with five or more nodes, and on the first test set the L_2-penalized network was slightly less accurate than OLS.

The two models also err differently (Fig. 7). The error of the polynomial oscillates across the whole interval and grows towards its ends, as a degree-12 polynomial bends to follow a narrow peak, and outside the data it diverges

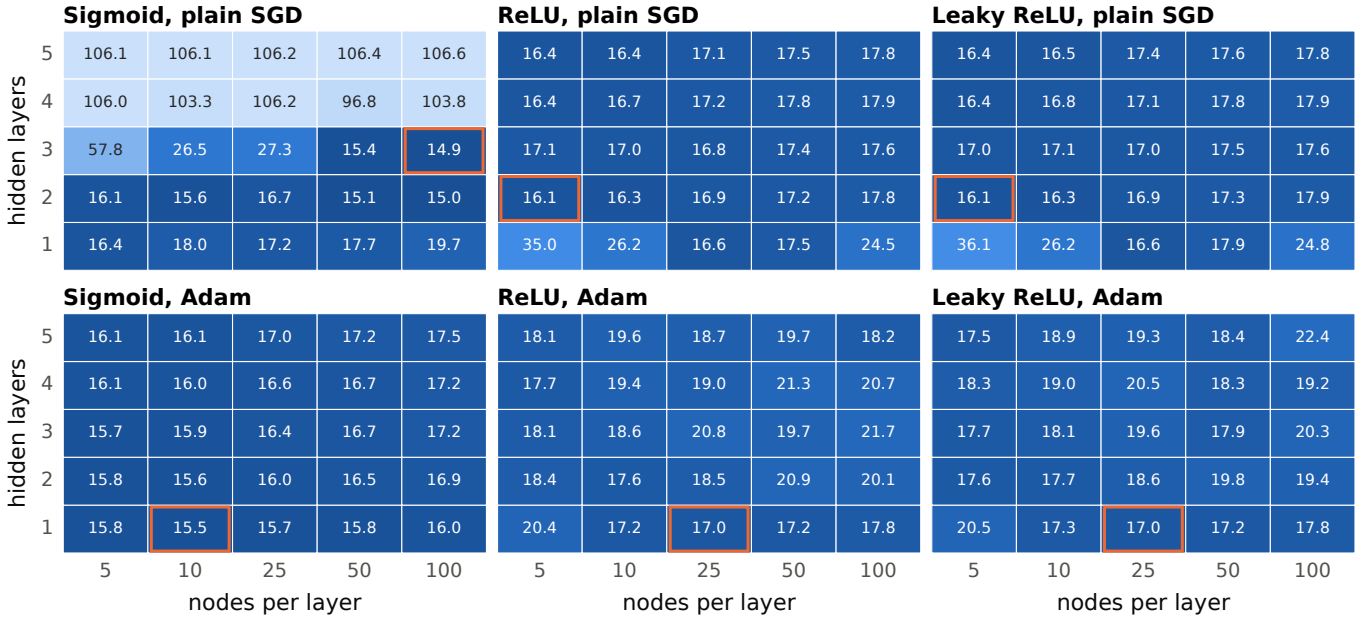


FIG. 3. With Adam all architectures fit about equally well and the sigmoid fits best, whereas plain SGD fails for sigmoid networks with four or more hidden layers. CV MSE ($\times 10^3$) against the number of hidden layers and nodes per layer after 3000 epochs, for the three activation functions with plain SGD (top) and Adam (bottom). Each cell is the best of three learning rates. Darker is better; the square marks the lowest value in each map. Predicting the training mean gives a CV MSE of about 106×10^{-3}.

TABLE II. The sigmoid networks with five or more hidden nodes deviate least from the true function, although all CV errors overlap within their standard errors. Selected models for the 100 observations of Runge's function, all errors $\times 10^3$; bold marks the lowest error in each column. CV errors are ten-fold, on the same folds for all models, with standard errors. Test errors use 2000 independent observations with noise variance 10.1×10^{-3}, and MSE_g is the error against the true function on the same inputs. For networks the test errors are the mean and standard deviation over ten initializations, and the fit time is for one fit on the 100 points. All networks use Adam, batches of 16 and the epoch with the lowest mean CV error, except the penalized and ReLU networks, which are trained for 3000 epochs.

Model	Settings	Parameters	CV MSE	Test MSE	MSE_g	Fit time
OLS	degree 12	13	17.3 ± 1.4	11.3	1.62	0.5 ms
Ridge	degree 12, $\lambda = 10^{-10}$	13	17.3 ± 1.4	11.3	1.62	0.1 ms
Lasso	degree 15, $\lambda = 10^{-4}$	16	19.2 ± 2.4	13.2	3.43	0.13 s
1-2-1 sigmoid	$\gamma = 3 \times 10^{-3}$, 4716 epochs	7	15.6 ± 1.6	11.2 ± 0.4	1.34 ± 0.46	2.9 s
1-5-1 sigmoid	$\gamma = 3 \times 10^{-3}$, 4716 epochs	16	15.7 ± 1.4	$\mathbf{10.5 \pm 0.2}$	0.60 ± 0.22	2.9 s
1-50-1 sigmoid	$\gamma = 3 \times 10^{-3}$, 943 epochs	151	15.3 ± 1.5	10.7 ± 0.5	0.85 ± 0.57	1.6 s
1-100-1 sigmoid	$\gamma = 3 \times 10^{-4}$, 7897 epochs	301	15.4 ± 1.5	$\mathbf{10.5 \pm 0.2}$	$\mathbf{0.58 \pm 0.24}$	6.9 s
1-50-50-1 sigmoid	$\gamma = 3 \times 10^{-3}$, 229 epochs	2701	15.0 ± 1.6	11.1 ± 0.5	1.12 ± 0.56	0.3 s
1-100-100-1 sigmoid	$\gamma = 3 \times 10^{-4}$, 744 epochs	10 401	$\mathbf{14.9 \pm 1.5}$	10.9 ± 0.9	1.06 ± 1.02	2.0 s
1-50-50-1, L_2	$\gamma = 10^{-3}$, $\lambda = 10^{-4}$	2701	17.1 ± 2.1	11.5 ± 0.3	1.71 ± 0.35	4.9 s
1-100-100-1, L_1	$\gamma = 3 \times 10^{-4}$, $\lambda = 10^{-4}$	10 401	16.2 ± 2.0	11.1 ± 0.4	1.31 ± 0.39	10.2 s
1-25-1 ReLU	$\gamma = 10^{-3}$, 3000 epochs	76	17.0 ± 1.5	11.1 ± 0.4	1.18 ± 0.42	1.5 s

within a short distance. The sigmoid networks level off instead, which keeps them bounded but not necessarily close to g.

5. *Approximation error and amount of data*

The noise-free fits indicate that the advantage of the networks comes from their approximation power rather than from the way they handle noise, although these fits also depend on the training (Fig. 8a). Without noise,

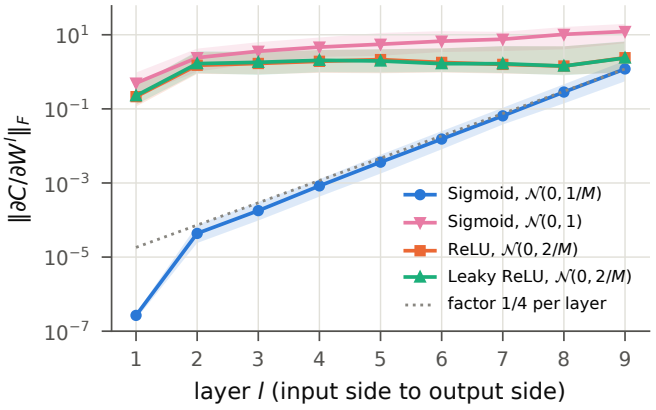


FIG. 4. The gradient vanishes towards the input of a deep sigmoid network, but not with ReLU or with weights drawn from $\mathcal{N}(0, 1)$. Frobenius norm of the weight gradient in each layer of an untrained network with eight hidden layers of 50 nodes, on the Runge data. Median and interquartile range over 30 initializations. The dotted line falls by a factor four per layer.

the error of the best polynomial fell by only about a half with every second degree, the rate set by the poles of g (Sec. II G), while two sigmoid nodes beat a polynomial with as many parameters by more than a factor of ten. For $\cos 3x$ the ranking was reversed: ten polynomial coefficients gave a mean-square error near 10^{-10}, far below what the networks reached.

The network builds the peak from two sigmoid steps of opposite sign whose sum is a localized bump (Fig. 9), whereas in the monomial basis the polynomial combines powers of x that extend over the whole interval, with terms up to several hundred times larger than the peak that nearly cancel.

With noise, the outcome depends on the amount of data (Fig. 8b,c). Here the polynomial was selected by cross-validation for each new data set, whereas the network kept the settings chosen on the main data set. For Runge's function, the network was closer to g than the polynomial in every one of 40 new data sets of 100 observations, but its lead shrank as n grew, and at 1000 observations the polynomial was more accurate in most data sets, even when the network was trained ten times longer. For $\cos 3x$, the polynomial was at least as accurate at every size, and from 500 observations on it was better in nearly every data set.

B. Classification of handwritten digits

1. *Optimizer and learning rate*

The optimizers behaved as for Runge's function, but every pass through 50 000 images has a noticeable cost (Fig. 10). After 200 passes at its best learning rate, plain GD, with one update per pass, was still below the accuracy that the minibatch methods reached in two or three. At its best rate plain SGD was within the standard error of Adam and RMSprop, which kept a high accuracy over a wider range of learning rates, centred about three hundred times lower.

2. *Width, depth and activation*

The width of the hidden layers mattered more than their number (Fig. 11a–c): widening a layer from 32 to 512 nodes gained about two percentage points, extra layers changed little, and the ReLU family was slightly ahead of the sigmoid. The widest networks reached almost 100% training accuracy while their validation accuracy, near 98%, still increased with the width (Fig. 11f), so a layer wider than the largest in the grid might do slightly better. The one-standard-error rule selected a single leaky ReLU layer of 512 nodes, and a ReLU layer of the same size was within its standard error.

The vanishing gradients seen for Runge's function reappeared on MNIST (Fig. 11d,e): with plain SGD, sigmoid networks with six or more hidden layers fell to the accuracy of guessing. Adam delayed this collapse, and ReLU networks trained at every depth up to ten layers.

3. *Input scaling, initialization and output cost*

Unscaled inputs and unscaled initial weights both reduced the accuracy (Fig. 12a). With raw grey levels from 0 to 255, almost every output of the first sigmoid layer was saturated, that is, below 0.01 or above 0.99, before and after training, whereas with scaled pixels none started there (Appendix C); even at its best learning rate this network ended five percentage points lower. Weights drawn from $\mathcal{N}(0, 1)$ saturated more than half of the outputs of both layers and cost about 3.5 percentage points, and the two errors together cost more than either alone.

A softmax output trained on the squared error mainly slowed the training (Fig. 12b): it lagged behind at small learning rates and in the first epochs, but at a large enough learning rate it ended close to the cross entropy. The softmax derivative scales the output error by the predicted probabilities (Appendix A), so for the nearly uniform predictions of the untrained network the output error, and with it the gradient, was about ten times smaller than with the cross entropy. The vanishing gradient of a confidently wrong prediction did not arise at the start, since the untrained network made no confident predictions (Appendix C).

4. *Penalties*

As for Runge's function, the penalties gave no measurable gain (Fig. 13), here with every setting stopped at its best epoch. Every penalty up to 10^{-5} was within one

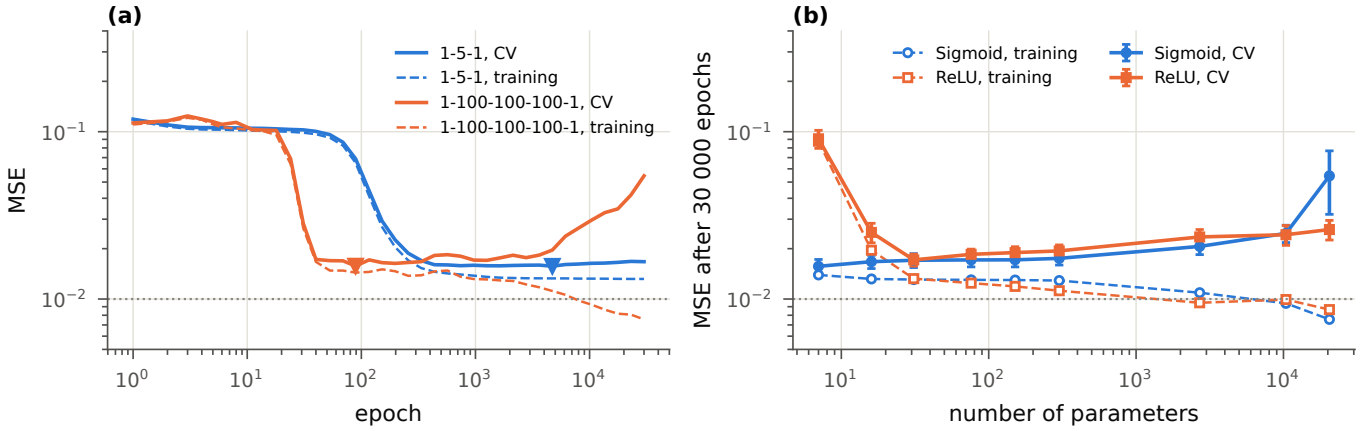


FIG. 5. Large networks overfit when trained for long, but early stopping removes most of the effect. Training without penalty, with Adam at $\gamma = 3 \times 10^{-3}$. (a) CV (solid) and training (dashed) MSE against epoch for a small and a large sigmoid network; triangles mark the epoch with the lowest mean CV error. (b) CV and training MSE after 30 000 epochs against the number of parameters, for networks from 1-2-1 to three hidden layers of 100 nodes.

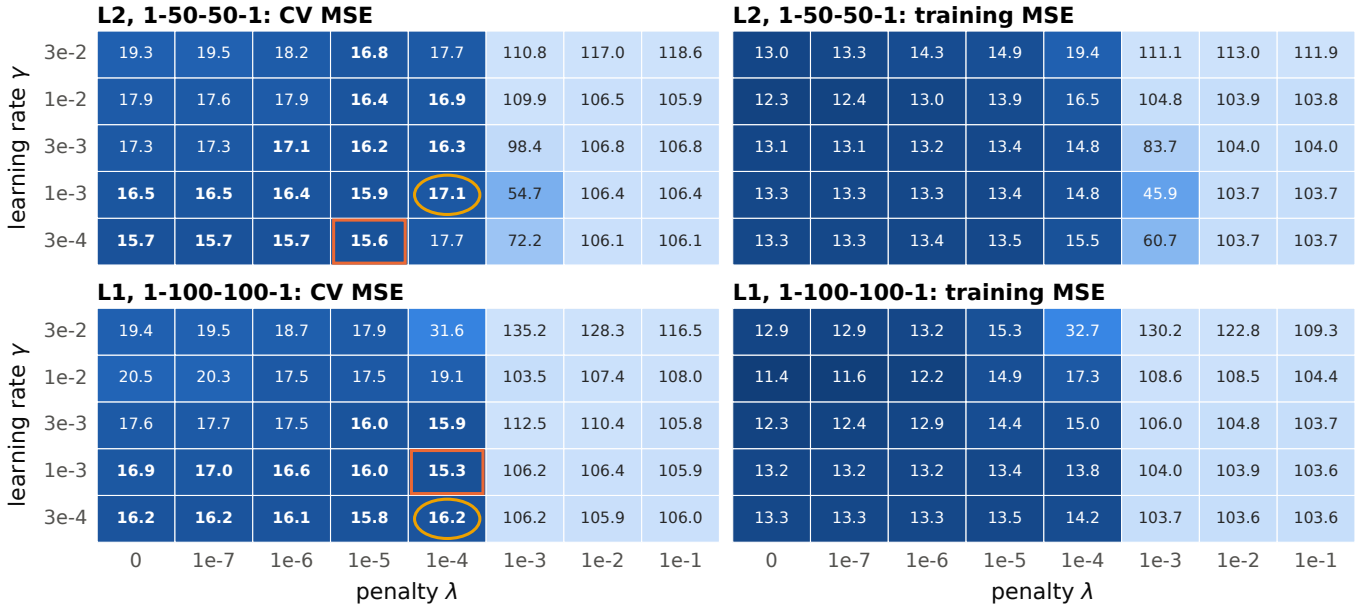


FIG. 6. Penalties up to 10^{-4} are indistinguishable from no penalty, and larger ones degrade or destroy the fit. CV MSE (left) and training MSE (right), $\times 10^3$, against the learning rate and the penalty after 3000 epochs, for the architecture with the lowest CV error for each penalty. Darker is better. The square marks the minimum, bold numbers are within one standard error of it, and the circle marks the choice of the one-standard-error rule. Predicting the training mean gives a CV MSE of about 106×10^{-3}.

standard error of no penalty, and a penalty of 10^{-3} cost about one percentage point with L_2 and four with L_1. The one-standard-error rule selected L_2 with $\lambda = 10^{-5}$ at $\gamma = 10^{-3}$, stopped at epoch 14, where the validation accuracy peaked.

5. *Test accuracy and remaining errors*

One hidden layer removed about 70% of the errors of logistic regression, many times more than the standard errors (Table III), and both models were within the standard errors of their scikit-learn counterparts. Over five initializations, the standard deviation of the test accuracy of the network, 0.1 percentage points, was smaller than the binomial standard error of a single run, 0.14, and its validation accuracy was within two standard errors of the test accuracy. Since scikit-learn has no leaky ReLU, we compared our network on the validation images with its ReLU network, trained on the same images with the same optimizer settings but its own initialization and no penalty on the biases; the two reached similar accuracies,

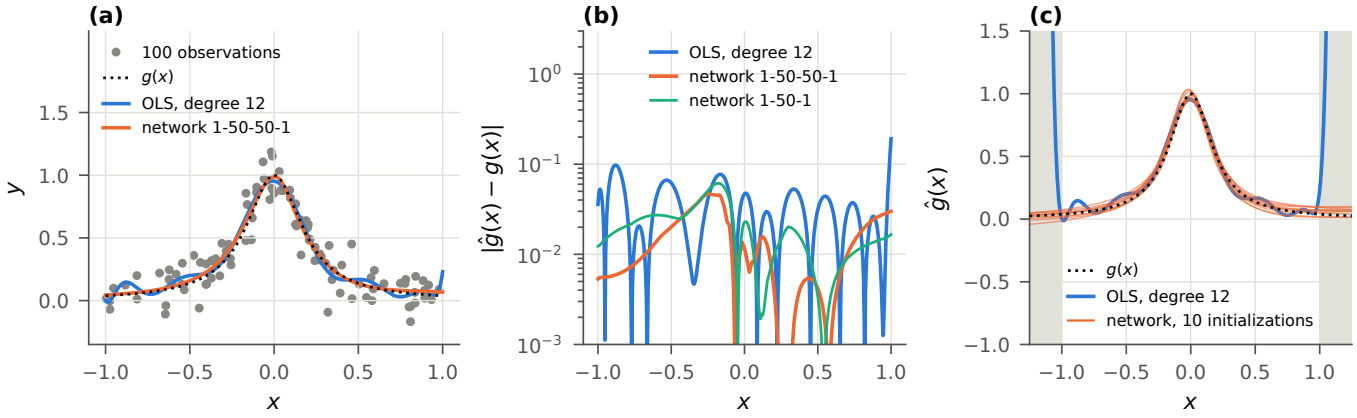


FIG. 7. The error of the polynomial oscillates across the interval and grows quickly outside the data, whereas the networks level off. OLS of degree 12 and sigmoid networks refitted on all 100 observations. (a) The fits, with the network shown as the median over ten initializations. (b) Distance to the true function inside the data range. (c) The fits on a wider interval; the shaded regions contain no data, and each thin line is one initialization of the 1-50-50-1 network.

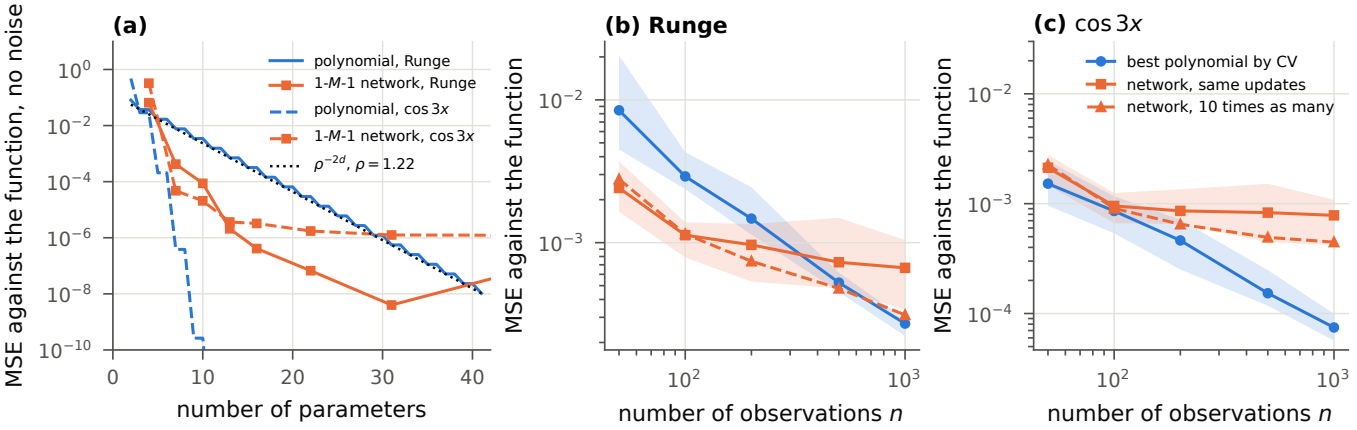


FIG. 8. Without noise, a few sigmoid nodes approximate Runge's function far better than a polynomial with as many parameters, but not $\cos 3x$; with noise, the advantage of the network shrinks as the data grow. (a) Mean-square distance to the target function without noise, against the number of parameters, for least-squares polynomials in the Chebyshev basis, fitted and evaluated on 2001 equally spaced points, and for 1-M-1 sigmoid networks, trained and evaluated on 401 points (best of five initializations), for Runge's function and for $\cos 3x$. The dotted line has the slope ρ^{-2d} of Sec. II G. (b, c) Error against the target function over 40 new data sets of each size, for the polynomial chosen by ten-fold CV over degree and Ridge penalty, and for a 1-50-1 network with the settings chosen on the main data set, trained with the same number of updates as there or with ten times as many. Lines are medians and bands interquartile ranges (no band for the longer training).

within the spread between initializations (Appendix C). This is a reference, not a check of the implementation, which Table I provides. Published results, obtained under different training conditions, are slightly higher for a dense network with 800 hidden nodes [13] and clearly higher for the convolutional network LeNet-5 [12].

The confusion matrix shows which digits remain difficult (Fig. 14a). The most frequent errors were fours read as nines, twos and fives read as threes, and sevens read as twos. Four and nine, and three and five, are among the pairs with the most similar mean images, but seven and two are not (Appendix C), so similar average shapes are consistent with some of these confusions but not others. Logistic regression made the same errors and many more, led by twos read as eights and sevens as nines; the hidden layer removed most of these, but only halved the confusion of four with nine, which became the most common error. Precision was lowest for 8 and 3, to which unclear digits were often assigned, and recall was lowest for 5 (Fig. 14b); the F_1 scores lie between 97 and 99%.

The network was less confident when it was wrong than when it was right: the median probability it gave a wrong prediction was 0.78, against more than 0.9999 for a correct one, which by itself says nothing about how well its probabilities are calibrated. Several of its most confident errors (Fig. 15, right) are digits that a human reader might also hesitate over.

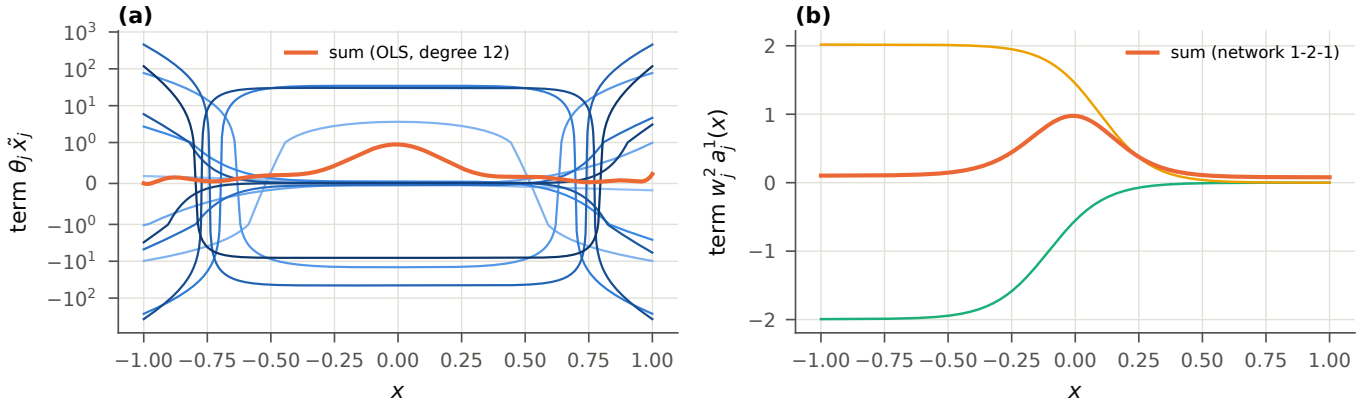


FIG. 9. The network builds the peak from two sigmoid steps, the polynomial from large global terms that nearly cancel. (a) The twelve terms $\theta_j \tilde{x}_j$ of the degree-12 OLS fit, with $\tilde{x}_j$ the standardized powers, on a symmetric logarithmic axis. (b) The two terms of a 1-2-1 sigmoid network, each an output weight times the output of a hidden node, and their sum. The constant terms are not drawn, but are included in the sums.

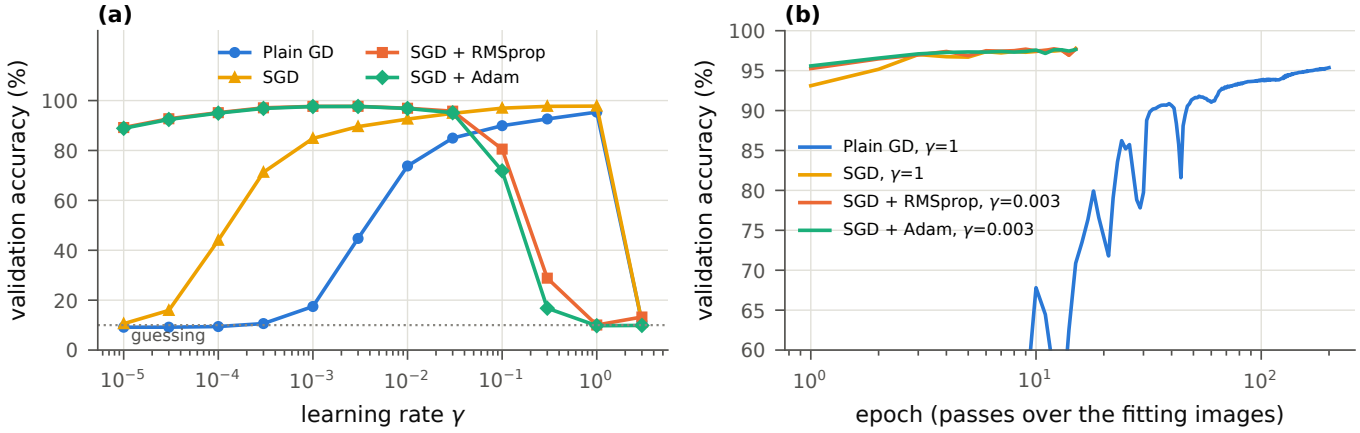


FIG. 10. On MNIST plain GD is far too slow, and the adaptive methods keep a high accuracy over a wider range of learning rates than plain SGD. One hidden layer of 128 ReLU nodes, with batches of 128. (a) Validation accuracy after 15 epochs (200 for plain GD) against the learning rate. (b) Validation accuracy against epoch at each method's best rate.

V. DISCUSSION

A. Learned versus chosen features

Whether learned features outperform chosen ones depended on how well the chosen basis suits the problem, not on the number of parameters. The poles of Runge's function limit polynomials to the slow convergence of Sec. II G (Fig. 8a), and a polynomial of degree 12 that follows the narrow peak oscillates across the interval and diverges outside the data, whatever basis represents it (Fig. 7); in the monomial basis, this shows up as large terms that nearly cancel (Fig. 9). Pairs of sigmoid nodes instead form localized bumps that match the peak. For $\cos 3x$, which a short polynomial expansion represents almost exactly, the chosen basis was far superior without noise, and with noise at least as accurate at every data size.

The advantage of the learned basis also shrank with more data: as n grows, cross-validation allows the polynomial a higher degree, and its error then fell faster than that of a network whose settings were tuned on the main data set only (Fig. 8b,c); a network retuned for each size might fare better. The OLS polynomial is moreover fitted in under a millisecond, whereas a network takes seconds per fit and has about ten settings to tune (Table II). For function fitting, a learned basis therefore seems worth its cost mainly when it suits the function better than the obvious one and the data are few.

For handwritten digits no comparable basis is known, and the weights of logistic regression show the limits of a model that is linear in the pixels (Fig. 15, left). Each digit is a single template against which an image is scored pixel by pixel, so a four with a closed top resembles a nine, and a stroke shifted by a few pixels falls on different weights. A hidden layer can combine pixels into intermediate features, plausibly parts of strokes and loops, although we did not inspect what its nodes represent. It removed

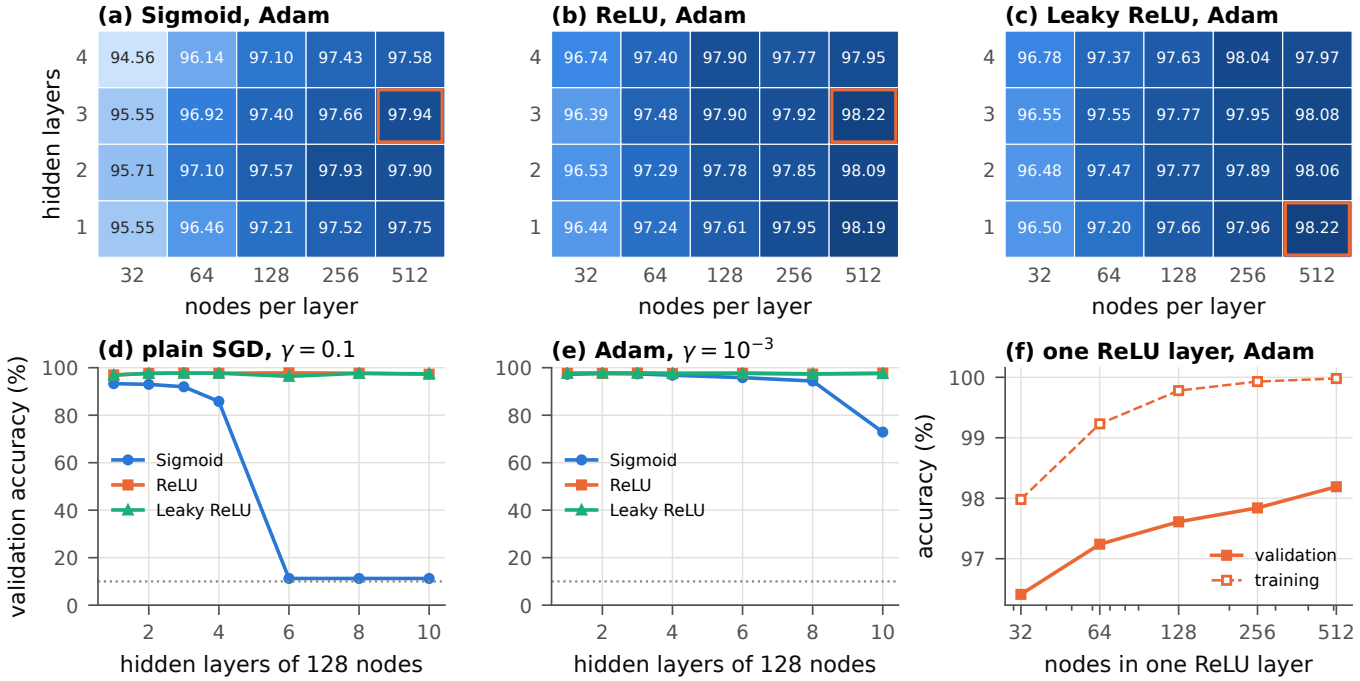


FIG. 11. The width matters more than the depth, and with plain SGD sigmoid networks fail from six hidden layers on, whereas ReLU networks train at every depth. Validation accuracy (%) on MNIST. (a–c) Against the number of hidden layers and nodes per layer for three activations, after 15 epochs of Adam at $\gamma = 10^{-3}$; darker is better, and the square marks the highest value. (d, e) Against depth for networks with 128 nodes per layer, with plain SGD and with Adam. (f) Validation and training accuracy against the width of one ReLU layer.

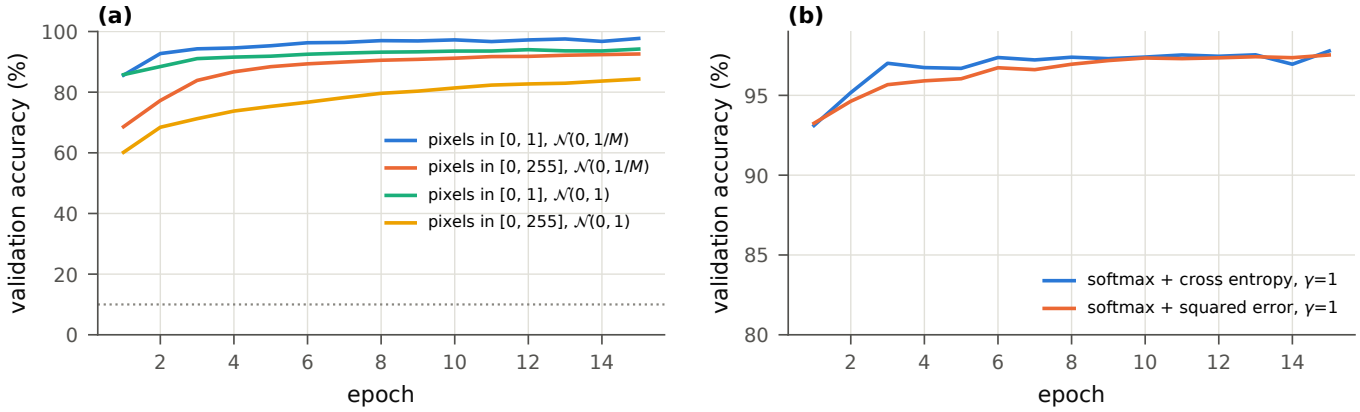


FIG. 12. Unscaled pixels and unscaled initial weights both lower the accuracy, whereas a softmax output trained on the squared error mainly slows the training. Validation accuracy against epoch with plain SGD; each curve is the best of three or more learning rates. (a) Two hidden layers of 128 sigmoid nodes, with pixels scaled to $[0, 1]$ or left in $[0, 255]$, and weights drawn with variance $1/M_{l-1}$ or 1. (b) One hidden layer of 128 ReLU nodes with a softmax output, trained on the cross entropy or on the squared error.

most of the errors of logistic regression, but it still treats every pixel position separately. Convolutional networks apply the same local filters at every position of the image and perform even better [12].

B. Training

In both problems, the way a network was trained mattered as much as its architecture. The adaptive optimizers found no better minima than a well-tuned plain SGD, but worked over a much wider range of learning rates, and Adam hardly depended on the batch size, which makes them the safer default when a search over

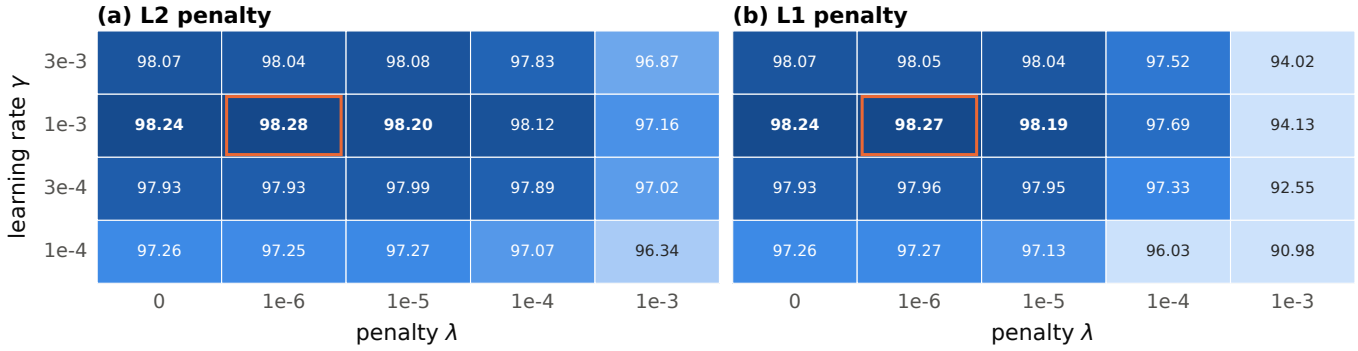


FIG. 13. Penalties up to 10^{-5} cost no measurable accuracy, and the largest ones lower it. Validation accuracy (%) of the 784-512-10 leaky ReLU network against the learning rate and the penalty, after up to 20 epochs of Adam. Darker is better; the square marks the highest value, and bold numbers are within one binomial standard error of it.

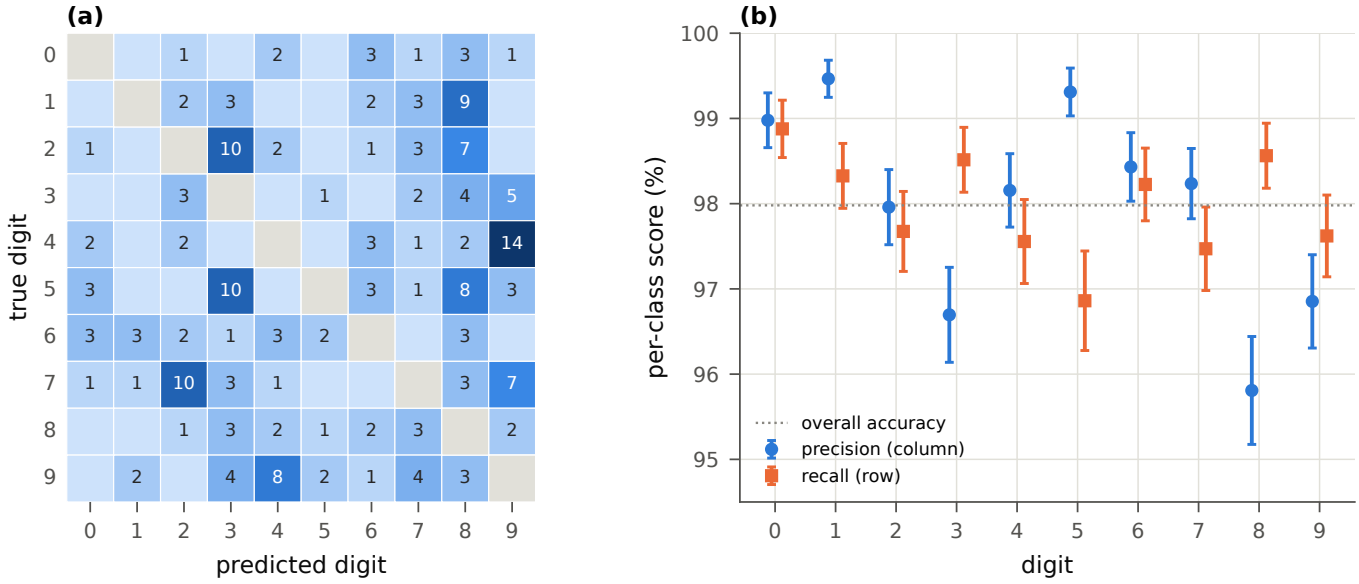


FIG. 14. The remaining errors concentrate in a few pairs of digits, led by fours read as nines. The selected 784-512-10 network on the test images. (a) Confusion matrix, rows true and columns predicted digit; the diagonal is left blank, empty cells are zero, and the colour shows the number of errors. (b) Precision and recall of each digit with binomial standard errors.

hyperparameters is costly. Their normalized steps also explain why Adam trained deep sigmoid networks that plain SGD could not. The gradient reaching the first layers shrinks by about a factor of four per layer (Fig. 4), as the largest sigmoid derivative of $\frac{1}{4}$ in Eq. (11) implies. Adam does not restore this gradient, but it rescales each step to a size of order γ, so that every layer moves, while plain SGD left the first layers practically unchanged (Appendix C). Moving every layer is not sufficient, however, since with ten sigmoid layers Adam still reached only 73% on MNIST. ReLU activations avoid the repeated factor of at most $\frac{1}{4}$, since their derivative is one on every active node, and ReLU networks trained at every depth we tried in both problems; inactive nodes or badly scaled weights can still make their gradients vanish or explode. The scaling of the inputs and initial weights matters for a related reason, as both decide whether the nodes start out saturated. Finally, matching the output activation to the cost gives a gradient proportional to the error itself, although on MNIST a mismatched pair mainly slowed the training.

C. Regularization and model complexity

Early stopping was the most effective form of regularization in both problems. On MNIST, where every penalty was combined with early stopping, the penalties added nothing measurable; on Runge's function, penalties trained for a fixed number of epochs did not improve on the early-stopped networks. There, networks from 7 to more than 20 000 parameters were all within one SE of the best when stopped at the right epoch, so the number of parameters says little about the complexity of a

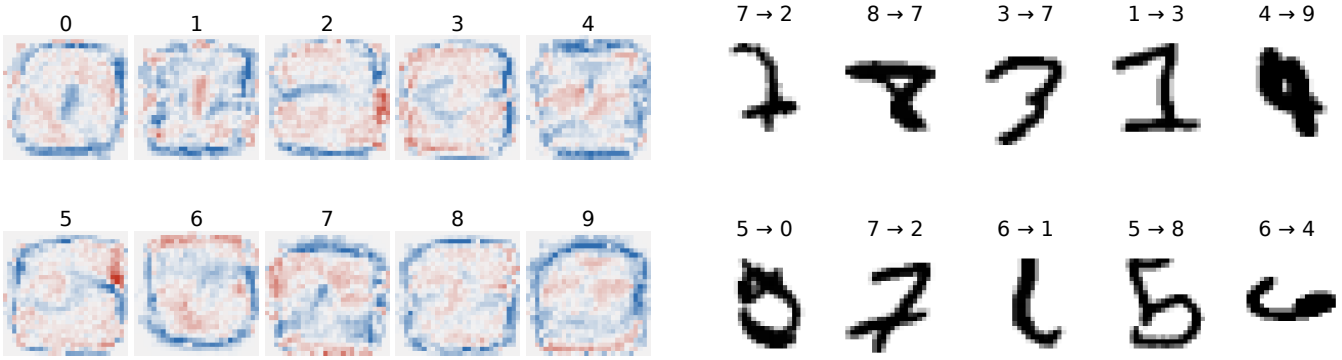


FIG. 15. Logistic regression scores each digit against a single pixel template, and several of the most confident errors of the network are ambiguous digits. Left: the weights of logistic regression for each digit, shown as 28×28 images (blue negative, red positive). Right: the ten test images that the selected network misclassifies with the highest confidence, labelled true → predicted.

TABLE III. One hidden layer removes most of the errors of logistic regression, and our implementations reach accuracies similar to those of scikit-learn. Test accuracy in percent on the 10 000 MNIST test images, used once, with binomial standard errors; bold marks the most accurate of our and scikit-learn's models. All models are fitted on the 60 000 training images. Our logistic regression (Adam with $\gamma = 10^{-3}$, L_2 penalty $\lambda = 10^{-7}$, 25 epochs) and 784-512-10 network (Adam with $\gamma = 10^{-3}$, $\lambda = 10^{-5}$, 14 epochs) use the settings chosen on the validation images, with batches of 128; the 784-100-10 network uses fixed, untuned settings (plain SGD, $\gamma = 0.1$, batches of 100, 20 epochs). Since scikit-learn has no leaky ReLU, its 784-512-10 network uses the sigmoid activation, with the same training settings but its own initialization and no penalty on the biases; a comparison with ReLU nodes on the validation images is given in Sec. IV B 5. Times are for one fit on a laptop and only indicate the order of magnitude. The published results are for a linear classifier and the convolutional network LeNet-5 [12], and for a network with 800 hidden nodes trained with the cross entropy [13].

Model	Parameters	Accuracy	Time
Guessing	0	10	
Our code			
Logistic regression	7850	92.77 ± 0.26	8 s
784-100-10 ReLU, SGD	79 510	97.67 ± 0.15	13 s
784-512-10 leaky ReLU	407 050	$\mathbf{97.98 \pm 0.14}$	6 min
scikit-learn			
Logistic regression	7850	92.57 ± 0.26	12 min
784-512-10 sigmoid	407 050	97.96 ± 0.14	10 min
Published			
Linear classifier		88.0	
784-800-10		98.4	
LeNet-5, convolutional		99.05	

trained network. A more useful diagnostic in these runs is how far training has moved the parameters from their small initial values. Early stopping limits this distance, and an L_2 penalty limits the distance from zero, which is similar when the initial weights are small [8, Sec. 7.8]. At their best epochs, eight of the nine networks had moved their parameters by about the same distance, which then grew with the overfitting (Appendix C). This is an association within our runs rather than a general measure of complexity, since the distance also depends on how the network is parameterized. The narrow peak of Runge's function needs some large weights, and for the two-node network a penalty of 10^{-3} would cost several times more than the whole data term (Appendix C), consistent with such penalties degrading or destroying the fit (Fig. 6).

D. Evaluation of the methods

Table IV summarizes the strengths and weaknesses of the methods as seen in this study. For a smooth function of one variable, polynomial regression remains the first method to try: OLS and Ridge have closed-form solutions, are fast and are easy to check, and lost only for a function with nearby singularities and few data. For images, the network is clearly worth its cost. Among the gradient methods, Adam and RMSprop were the safest default, and Adam also trained much deeper sigmoid networks than plain SGD.

VI. CONCLUSIONS AND OUTLOOK

Learned features are worth their cost when the right features are unknown, or when the obvious ones represent the problem poorly. A feed-forward network with a handful of sigmoid nodes approximated Runge's function more accurately than the best polynomial on the same data, because the poles of the function make powers of x an inefficient basis. For a function without such poles, or with more data, the polynomial was both more accurate

TABLE IV. Strengths and weaknesses of the methods, as seen in this study.

Method	Strengths	Weaknesses
Polynomial regression (OLS, Ridge, Lasso)	Closed-form fit (OLS, Ridge) in under a millisecond; convex, with a unique solution for a full-rank design; best for smooth functions without nearby singularities and, here, for larger data sets	Features must be chosen by hand; slow convergence for Runge's function; large coefficients that nearly cancel; diverges outside the data
FFNN for regression	Learns a local basis; 1.5 to 2.8 times smaller error against Runge's function; bounded outside the data	Seconds per fit; about ten settings to tune; results vary between initializations; no closed-form solution to check against
Logistic regression	Fast; convex, so every minimum is global; weights viewable as digit templates	Linear in the pixels, so it cannot combine pixels into features
FFNN for classification	Learns features from the pixels; removes about 70% of the mistakes of logistic regression	Fifty times more parameters; learning rate, width and stopping point must be tuned; does not know that a shifted stroke is the same stroke
Plain GD	Simple; uses the exact gradient	One update per pass over the data; far too slow on MNIST
Plain SGD	Many cheap updates; within the error bars of Adam when tuned	Narrow range of good learning rates; fails for deep sigmoid networks
RMSprop and Adam	Work over a wide range of learning rates; Adam (the only one tested) trains much deeper sigmoid networks than plain SGD	Extra settings; no better minimum than a well-tuned SGD

and far cheaper than a network with fixed settings. For handwritten digits, where no good features are known, a single hidden layer removed most of the errors of logistic regression.

Training required as much care as the choice of architecture. Plain SGD failed for deep sigmoid networks because the gradient vanishes towards the input; ReLU activations avoided this problem in the networks we tried, and the adaptive steps of Adam largely compensated for it, while scaled inputs and initial weights kept the nodes out of saturation. Overfitting resembled that of polynomial models and was almost entirely removed by early stopping; L_1 and L_2 penalties did not improve on it, just as Ridge and Lasso added little to a polynomial of the right degree.

These conclusions have clear limits. The comparison with polynomials rests on one set of 100 observations, on which cross-validation could hardly separate the models; only the known function and large independent test sets did, and real problems offer neither. The study over many data sets used network settings tuned on one data set, the grids were coarse and the MNIST choices rest on one validation split, so differences of a tenth of a percentage point are not resolved, and deep sigmoid networks failed only for the learning rates we tried. A few unsaved pilot runs that sized the grids also printed errors on the first regression test set, which is therefore not strictly untouched; the models were selected on the cross-validation errors alone, and the second test set, read only by the final evaluation, gives the same conclusions.

Two extensions would test the main conclusion most directly. If our explanation is right, Runge's function in two or more dimensions, where the number of polynomial terms grows quickly with the degree, should move the balance further towards networks. On MNIST, a convolutional network trained with the same code and validation protocol would show how many of the remaining errors, such as fours read as nines, come from treating every pixel position separately.

Appendix A: Matched output activations and costs

Equation (7) multiplies the derivative of the cost by the derivative of the output activation. With a linear output $f' = 1$, so Eq. (2) gives $\boldsymbol{\delta}^L = (\boldsymbol{a}^L - \boldsymbol{y})/n$ directly. With a sigmoid output, $\sigma'(z^L) = a^L(1 - a^L)$ cancels the denominator in Eq. (3), and again $\delta^L = (a^L - y)/n$.

The softmax is not elementwise, because each output depends on all the scores: $\partial a_k^L/\partial z_j^L = a_k^L\left(\delta_{kj} - a_j^L\right)$, where δ_{kj} is the Kronecker delta. The chain rule with Eq. (4) then gives

$$\begin{aligned}\delta_j^L &= \sum_k \frac{\partial C_{\mathrm{CE}}}{\partial a_k^L}\frac{\partial a_k^L}{\partial z_j^L} = -\frac{1}{n}\sum_k \frac{y_k}{a_k^L}\, a_k^L\left(\delta_{kj} - a_j^L\right) \\ &= \frac{1}{n}\Big(a_j^L \sum_k y_k - y_j\Big) = \frac{1}{n}\left(a_j^L - y_j\right), \end{aligned} \tag{A1}$$

because the one-hot target has $\sum_k y_k = 1$. Consider a confidently wrong prediction, with $a_m^L \approx 1$ for a wrong class m and $a_c^L \approx 0$ for the true class c. It gives errors of size $1/n$ on both nodes, so the network receives a strong signal to correct itself.

TABLE V. The measurements behind the explanations in the text. The relative change of a layer is $\|\boldsymbol{W} - \boldsymbol{W}_0\|/\|\boldsymbol{W}_0\|$ in the Frobenius norm, with $\boldsymbol{W}_0$ its initial weights, $\|\boldsymbol{\theta} - \boldsymbol{\theta}_0\|$ is the Euclidean norm over all parameters, and a sigmoid output counts as saturated when it is below 0.01 or above 0.99, where its derivative is below 0.01.

Explanation	Measurement	Result
Plain SGD hardly changes the first layers of a deep sigmoid network (Sec. IV A 2)	Relative change of each layer of a network with five hidden layers of 50 sigmoid nodes after 3000 epochs on all 100 observations; median of ten initializations, at each learning rate of Fig. 3	Plain SGD: at most 0.4% in the first layer and 11–57% in the output layer; the training MSE stays at that of predicting the mean. Adam: every layer changes by 0.2 to 15 times its initial size
Adam moves every layer of a deep sigmoid network (Sec. V B)	Relative change of each layer after 15 epochs on MNIST, ten hidden layers of 128 sigmoid nodes, settings of Fig. 11d,e	Plain SGD: 14% in the last hidden layer, falling by a factor of about four per layer to below 0.003% in the first four layers. Adam: every layer changes by 1.3 to 3.8 times its initial size
Unscaled pixels and weights saturate the sigmoid (Sec. IV B 3)	Share of saturated outputs on the validation images, for the networks of Fig. 12a at their best learning rates	Scaled pixels and weights: none before training. Raw pixels: 96% of the first layer before and 98% after training. Weights from $\mathcal{N}(0, 1)$: 63% and 57% of the two layers. Both: nearly all of the first layer
No prediction is confidently wrong at the start (Sec. IV B 3)	Largest output probability of the untrained 784-128-10 network on the validation images	Median 0.16 and maximum 0.35, so no prediction is confidently wrong
Similar average shapes are consistent with some confusions (Sec. IV B 5)	Cosine similarity of the mean training images of each pair of digits, ranked among the 45 pairs	4 and 9 first, 3 and 5 third, 2 and 3 twelfth, 7 and 2 35th
Our network reaches an accuracy similar to scikit-learn's (Sec. IV B 5)	Validation accuracy of our selected network and of scikit-learn's with ReLU nodes, both trained on the 50 000 fitting images with the selected settings; five initializations each	$97.98 \pm 0.22\%$ and $97.95 \pm 0.13\%$ (mean and standard deviation)
In these runs, the distance moved tracks overfitting better than the number of parameters (Sec. V C)	$\|\boldsymbol{\theta} - \boldsymbol{\theta}_0\|$ along the curves of Fig. 5, mean over the folds	At the best epoch 7–10 for eight of the nine networks (7 to 20 501 parameters) and 19 for 1-100-1, whose CV error hardly changes after epoch 300; after 30 000 epochs from 7 to 200, growing with the overfitting
Large weights make a penalty costly (Sec. V C)	$\lambda\|\boldsymbol{\theta}\|^2$ of the two-node network of Fig. 9, against the data term of its cost, half its training MSE	Input weights about 5, output weights about 2; at $\lambda = 10^{-3}$ the penalty, 0.06, is eight times the data term, 0.007

The same softmax output with the squared error keeps the derivative of the softmax,

$$\delta_j^L = \frac{1}{n}\, a_j^L \Big[\big(a_j^L - y_j\big) - \sum_k a_k^L \left(a_k^L - y_k\right) \Big]. \qquad \text{(A2)}$$

For the confidently wrong prediction above, $\delta_c^L \approx 0$ because $a_c^L \approx 0$, and $\delta_m^L \approx 0$ because the bracket vanishes. The same happens for a sigmoid output with the squared error, where $\delta^L = (a^L - y)\, a^L(1 - a^L)/n$. The gradient vanishes exactly where it is needed most. The factor a_j^L also weakens the gradient in less extreme cases: for the uniform predictions $a_k^L = 1/K$ of an untrained network the sum in the bracket is zero, and the output error is K times smaller than with the cross entropy.

Appendix B: Penalties and the size of the weights

Figure 16 shows what the two penalties do to the weights of a trained 1-50-1 network. We use $\lambda = 10^{-3}$, ten times the value chosen by the one-standard-error rule, because it shows the effect clearly. Without a penalty the output weights spread over three orders of magnitude. Both penalties leave two large weights, so with this much shrinkage two nodes carry the fit. At this strength the L_2 fit is clearly degraded, with a training MSE of 36×10^{-3} for one initialization against 14×10^{-3} without penalty, whereas the L_1 fit is only slightly worse, at 16×10^{-3}. Two nodes is also the size that cross-validation found to be sufficient in Sec. IV A 4. The penalties differ in what they do to the rest. L_2 shrinks the other 48 weights to a few hundredths, while L_1 pushes them all down to about

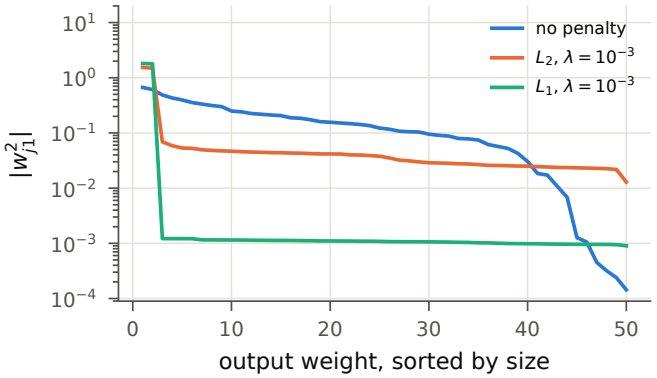


FIG. 16. Both penalties leave two large output weights, and L_1 pushes the others further towards zero than L_2. Output weights of the 1-50-1 sigmoid network after 3000 epochs of Adam on all 100 observations, sorted by size; median over ten initializations.

10^{-3}. They do not become exactly zero, because a subgradient step moves a weight past zero rather than onto it. This resembles the Lasso, which sets small polynomial coefficients to zero, while Ridge only shrinks them.

Appendix C: Measurements behind the explanations

Several explanations in Secs. IV and V are tested with the measurements in Table V. They use the training and validation data only. Where the networks are those of a figure, they are rerun with the same settings and seeds, and the reruns reproduce the saved curves of that figure exactly.

Appendix D: Use of AI/LLM tools

This declaration follows the course guidelines [22]. For report text the guidelines allow only level 0 (none) and level 1 (language only: spell-check, grammar, minor phrasing); any larger contribution to the writing is not permitted. For code the guidelines define five levels, 0 (none) to 4 (the model wrote most of a file, which the author then adapted, tested and commented).

1. Tools used

Claude Opus 5.5 (Anthropic), through the Claude Code desktop application, November 2026, for everything listed below. It worked directly in the project folder: it read the course material, wrote and ran code, inspected the results and wrote text. OpenAI Codex, through the Codex desktop application, in November 2026 reworked parts of the new code for readability only, as marked in the file rows below. The files reused from the earlier study also contain work by OpenAI Codex (October 2026), as declared in that report [10].

2. How the work was produced

I produced this report and its code during the project month, working with Claude as my coding and language tool. I derived the equations of Sec. II and Appendix A by hand and checked them against Chapters 4, 5 and 8 of the lecture notes before typing them in. I wrote the core of `network.py` — the activations, the costs, the feed-forward pass, the backward pass and the penalties — reusing the optimizers of our earlier study unchanged, and checked each piece against the tests of Table I as I wrote it. Claude helped me debug the backward loop, and drafted the parallelisation, plotting and notebook boilerplate, plus a few small helpers. I chose the grids, batches and folds, the one-standard-error rule, the panels of every figure and the models carried to the test sets, and I ran every experiment listed in Sec. IV. The text is my own writing: early passages were drafted with Claude as the tool, and I rewrote each section in my own words, checking every number against the files under `results/`. Claude's final audit of the report is described under Errors below. Near the end, Codex reworked parts of the new code for readability: it shortened loops and made the parameter handling simpler and more explicit. No experiment was rerun because of it, and the tests and the saved results were unchanged.

Claude also organised the LaTeX source file and formatted all tables and figures to make the report readable. I used this help because I cannot write LaTeX myself.

3. Text

Section	Level	Notes
Abstract	1	Written from my notes on the results; language checked.
Introduction	1	My own writing; language checked.
Theory	0	Equations derived by hand first, then typed; written independently.
Methods	1	My own writing; language checked.
Results, captions, tables	1	Written from my summaries of the runs; every number read from `results/`.

Discussion	1	My own writing; language checked.
Conclusions	1	Written from my notes on the results; language checked.
Appendices A and B	0	Derivation and analysis done by hand; the expressions in A are tested numerically against automatic differentiation.
This appendix	1	Structure follows the guidelines; my own words.
References	1	The list was compiled with Claude and each entry checked against Crossref or the publisher's page; I read the sources I cite.

File	Level	Role of the LLM
`network.py`	2	The activations, costs, feed-forward pass, backward pass and penalties are mine; Claude helped debug the backward pass and supplied a few helpers (the training wrap, `one_hot`, `accuracy` and the layer diagnostics); OpenAI Codex (November 2026) reworked the parameter unpacking, layer evaluation and gradient assembly for readability. The file-level declaration does not apply to every function.
`optimizers.py`	3	Reused unchanged from the earlier study, which declared it at this level: the update rules, gradients and cost functions are mine; the class structure and driver loops were drafted with Claude there (declared in [10]).
`estimators.py`	2	The estimator interface and rules are mine; the endpoints drafted with Claude; OpenAI Codex (November 2026) reworked the update rule and the two fit methods for readability.
`mnist_data.py`	2	The loading, splitting and scaling rules are mine; the archive download with its caching and checksum drafted with Claude.
`runge_experiments.py`	2	The grids, folds, one-standard-error rule and protocol are mine; the parallelisation and small helpers drafted with Claude.
`mnist_experiments.py`	2	The splits, settings and evaluation protocol are mine; the parallelisation and probes drafted with Claude.
`verification.py`	2	The checks follow my derivations and the course reference; the standalone autograd cost and the finite-difference loop drafted with Claude.

4. Code

Paths are relative to `Code/`. Every function with LLM assistance carries an `LLM-assisted` tag in its docstring or in the docstring of its module, stating the role; reused files keep the tags they already had, and a reused file carries the level the earlier study declared for it.

`regression.py`	4	Reused unchanged from the earlier study, which declared it at this level: generated with Claude there, with `checked_lasso_path` by OpenAI Codex (declared in [10]).
`plot_style.py`	4	Reused unchanged from the earlier study, which declared it at this level: the style module was generated with Claude there. Only `load_results` is new in this project, drafted with Claude.
`figures.py`	2	Panel choices and what each figure shows are mine; Claude drafted small helpers among the definitions: the tick labels, the annotated heat map and the panel letters.
`tests/` *(Project 1)*	4	`conftest.py`, `test_optimizers.py` and `test_regression.py` reused unchanged from the earlier study, which declared them at this level.
`tests/` *(new)*	2	The four test files new in this project: the expected values come from my derivations and the course reference; the test scaffolding drafted with Claude.
Notebooks	2	The cells call the modules; layout and the notes above assisted cells drafted with Claude.

5. Errors found during the work

These errors were caught by the tests, by reruns, or by reading the course material. A first test of logistic regression compared with a run of Adam that had not settled; it was replaced by plain gradient descent, which agrees with the reference. The first plan for the experiments missed requirements of the later weekly material and was changed after that material was read. A scikit-learn call with an outdated argument order stopped the final regression run after the networks had been scored on the test observations; the call was corrected and the same deterministic run repeated, with no change to any setting. The final audit found that the scikit-learn network in Table III was labelled leaky ReLU although the code gives it the sigmoid activation, because scikit-learn has no leaky ReLU, and that several numbers in the text did not match the saved results; both were corrected without rerunning any experiment. The measurements added to back the explanations and the comparison with scikit-learn's network on the validation images (Sec. IV B 5) use new code, the two `mechanisms` functions and `library_validation`, which changes no earlier result.

[1] M. Hjorth-Jensen, Machine Learning and the Physical Sciences: From Discriminative Methods to Generative Methods and Reinforcement Learning (2026), lecture notes for FYS-STK3155/4155, University of Oslo, accessed November 7, 2026.

[2] X. Glorot and Y. Bengio, in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, Proceedings of Machine Learning Research, Vol. 9 (2010) pp. 249–256.

[3] K. He, X. Zhang, S. Ren, and J. Sun, in *2015 IEEE International Conference on Computer Vision (ICCV)* (2015) pp. 1026–1034.

[4] V. Nair and G. E. Hinton, in *Proceedings of the 27th International Conference on Machine Learning* (Omnipress, 2010) pp. 807–814.

[5] A. L. Maas, A. Y. Hannun, and A. Y. Ng, in *ICML Workshop on Deep Learning for Audio, Speech and Language Processing* (2013).

[6] T. Tieleman and G. Hinton, Lecture 6.5 – RMSprop: divide the gradient by a running average of its recent magnitude (2012), cOURSERA: Neural Networks for Machine Learning.

[7] D. P. Kingma and J. Ba, in *3rd International Conference on Learning Representations (ICLR)* (2015) arXiv:1412.6980.

[8] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning* (MIT Press, Cambridge, MA, 2016).

[9] M. A. Nielsen, *Neural Networks and Deep Learning* (Determination Press, 2015).

[10] R. R. Øzdber, Regression, resampling and gradient descent for the Runge function (2026), report, Department of Physics, University of Oslo.

[11] Y. LeCun, C. Cortes, and C. J. C. Burges, The MNIST database of handwritten digits (2010), online database.

[12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, Proc. IEEE **86**, 2278 (1998).

[13] P. Y. Simard, D. Steinkraus, and J. C. Platt, in *Seventh International Conference on Document Analysis and Recognition* (2003) pp. 958–963.

[14] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, J. Mach. Learn. Res. **12**, 2825 (2011).

[15] L. N. Trefethen, *Approximation Theory and Approximation Practice*, extended ed. (SIAM, Philadelphia, 2019).

[16] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, *et al.*, Nature **585**, 357 (2020).

[17] D. Maclaurin, D. Duvenaud, and R. P. Adams, in *ICML 2015 AutoML Workshop* (2015).

[18] J. D. Hunter, Comput. Sci. Eng. **9**, 90 (2007).

[19] M. L. Waskom, J. Open Source Softw. **6**, 3021 (2021).

[20] M. Hjorth-Jensen, Cross-validating a neural network with scikit-learn: worked examples (2026), course material for FYS-STK3155/4155, University of Oslo, accessed November 7, 2026.

[21] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. (Springer, New York, 2009).

[22] FYS-STK3155/4155, Guidelines for declaring the use of large language models in project reports (2026), university of Oslo, accessed November 7, 2026.